

Alati za razvoj softvera predavanja

Čas 1

Informacije o kursu

- Kolokvijuma neće biti
- Neće biti kačeni snimci redovno
- Vežbe će se snimati
- Većina alata će biti vezana za C++/QT, ili pak opšteg karaktera (docker, analizatori koda)
- Projekti do kraja aprila
- Projekti
 - ne ocenjuje se funkcionalnost već samo korišćenje alata
 - isti broj ljudi po projektu kao na RS
 - u slučaju da je odbranjen RS, preporuka je prijaviti se projektu otvorenog koda
 - moguće je unaprediti projekat iz RS uz pomoć alata
 - napisati kratak izveštaj o projektu
 - saveti za opensource:
 - ako radiš opensource, dodati u izveštaj koje su popravke napravljene, da li ih je tim primio, i sl.
 - naći nešto u čemu radimo svakodnevno
 - ne birati stvari za koje treba veliko domensko znanje (kompajleri, baze, i sl.)
 - ako BAŠ ne možemo da nađemo, možemo da mu napišemo interesovanja, pošaljemo mejl, pa će on pokušati da nađe projekat za nas
- Literature nema, knjige iz opisa kursa ne pokrivaju ništa što ćemo mi raditi i zastarele su osim malo one o build sistemima
- Postavljaće se posebno materijale za svaki čas/tehnologiju
- Mogući primer na ispitu:
 - Dobijemo program koji ima bag ili curenje u memoriji, gde je zadatak ispraviti bag/curenje
- Raspodela poena još ne postoji
- Linux shell se podrazumeva
 - `!$` - poslednji argument prethodne komande

Zanimljiv alat:

<https://bigbluebutton.org/>

Git

- Problemi sa ručnim bekapovanjem svake verzije:
 - zauzimanje memorije
 - ne znamo gde je šta izmenjeno, možda ima mnogo bekapa koje opisuje samo redni broj / datum / ime, ne znamo gde tačno da se vratimo ako je nastao bag
 - `bkp20200910-001-implemented-main` - primer naziva bekapa projekta

- Čuvanje izmena u aktuelnom fajlu koji editujemo (libre office)
 - u slučaju korumpiranja, bekapi se prljaju
- Rešenje 1:
 - Čuvanje samo izmena između verzija projekta:

```
v1 - diff v1 v2
v2 - diff v2 v3
v3 - diff v3 v4
v4 - trenutna verzija
```

- Problemi:
 - ako izgubimo neki diff između nećemo moći da vratimo sve verzije pre njega
 - ako više ljudi radi na projektu, sinhronizacija između njih i njihovih verzija

Zanimljivost:

<https://archiveprogram.github.com/>

- Rešenje 2: Subversion (svn)
 - Problemi:
 - revizije su se čuvale samo na serveru, trebala je konekcija da se vide razlike/revizije
 - server je single point of failure za istoriju projekta (aktuelne verzije postoje kod korisnika)
- Rešenje 3: Distribuirani sistemi (git)
 - svaki dev ima svoju verziju i celu istoriju
 - centralni server isto pamti celu istoriju i aktuelnu verziju
 - mora se beležiti drukčije label-i verzija, da se ne bi došlo do situacije da dve verzije imaju isti naziv
 - Problemi:
 - rrešeni

Čas 2

Git - Nastavak

- Problem sa serverskom arhitekturom (svn):
 - ako dev nema internet, ne može da uradi ništa jer ne može da komunicira sa serverom
- Ovo se rešava distribuiranim sistemom za praćenje revizija
 - svaki korisnik kod sebe ima celu istoriju projekta
 - sve se radi lokalno, pa se šalje nova verzija serveru
 - moguće je lokalno verzionisanje
 - problem divergencije ako se vuku izmene od mnogih peer-ova
 - bez obzira što je sam sistem implementiran distribuirano, postojaće skoro uvek server (npr Gitolite u firmi, GitHub veb sajt), tako da će sinhronizacija biti izvršena samo sa centralnim serverom
 - server javlja svima koji šalju izmene da li postoji kolizija između serverskog projekta i projekta uvezenog od konkretnog peer-a

- minimizuju se komunikacija sa serverom, samo pullanje, pushanje i komunikacija pri razrešavanju kolizija uključuju server

Git - alat za verzionisanje

GitHub, GitLab - hosting servisi

- do kolizije dolazi kad pokušavamo serveru da pošaljemo reviziju fajla koji je u međuvremenu (od našeg poslednjeg pulla ili od kad smo branchovali) već revidiran od strane nekog drugog i pushovan na server
- Neke komande:
 - `git init` - inicijalizuje git repo u trenutnom direktorijumu
 - `git add filename` - dodavanje izmena u staging area
 - **staging area** - deo koji sistem za verzionisanje prati i koji se commituje komandom `commit`, praktično zakazivanje izmena da budu deo sledećeg commita
 - **working area** - izmene koje nisu premeštene u staging area
 - bitno: pričamo o izmenama, a ne o fajlovima, neke izmene nad istim fajlom se mogu dodati u staging area, a druge ostaviti u working area
 - `git rm --cached filename` - uklanjanje fajla iz staging area
 - `git status` - ispisuje šta se trenutno dešava, koji fajlovi su menjani, dodati, ili obrisani od poslednje verzije, koji nisu ni dodati u staging area, ako ima commitova koliko ih nije pushovano, ako smo usred mergea gde su konflikti, i slično
 - `git config --global` je isto sto i menjanje `.gitconfig` fajla u home direktorijumu repoa
 - `git status` - pregled trenutno nekomitovanih i izmenjenih fajlova
 - `git commit` - ne commituje u slučaju da ne napišemo deskripciju, dobra praksa je da se piše naslov, prazan red, poruka o izmenama ukratko, prazan red, detaljniji opis izmena
 - `git commit -m 'Naslov ovde'` - u jednom redu commitovanje gde se ne otvara editor da se piše poruka već je on automatski doda ovde, može se koristiti više `-m` tako da se doda tekst opisa commita i novi paragrafi
 - `git commit -m 'Naslov' -m 'kratka poruka' -m 'detaljnije objašnjenje'`
 - `git log` - pregled istorije commitova
 - **commit id** - "kobasica" heksadekadnih cifara za identifikaciju commitova
 - koriste se nekakve hash sume da se generiše **commit id** jer bi postojao problem sa usklađivanjem enumeracije između više korisnika koji možda nisu up to date sa serverskom verzijom
 - brojevi verovatno jedinstveni na celom svetu, zasigurno u okviru projekta
 - `git diff` - izmene od poslednjeg commita
 - napomena: ovo se odnosi na izmene koje su u working area, kada se jednom doda fajl u staging area komandom `git add` ne prikazuje se više uopšte preko `diff`
 - `git diff id_commita` - prikazati razlike od zadatog commita do trenutne verzije (SVE izmene, bez obzira da li su u working ili staging area)
 - git će popuniti sam id ako napišete samo prvih nekoliko cifara (bez korišćenja taba), ako slučajno ima više commitova koji počinju tim ciframa u istoriji bira skoriji
 - `git diff HEAD` - diff od poslednjeg commita
 - `HEAD`
 - `HEAD~1`

- `HEAD~2`
 - itd.
 - `git diff HEAD~3 HEAD~2`
- `git commit -amend` - [modifikovanje poslednjeg commit-a](#)
- `.git/objects` - folder u kome se čuvaju commitovi
- `git show comit_id`
- `HEAD` - poslednja revizija (ne poslednje izmenjeno, vec commitovano), moze se razumeti kao pokazivac na niz commit-ova
- `HEAD~1` - preposlednja revizija
- za sve postoji [zakrpa](#)
- moguće u `.gitconfig` fajlu definisati aliase
 - moguće je na ispit doneti spisak svojih alijasa

Čas 3

GDB

- program se uključuje u blokiranom stanju, tako da možemo da definišemo breakpoints itd. pre nego što započnemo izvršavanje
- `run` ili `r` - pokreće program (uvek ispočetka)
- `cont` ili `c` - nastavi izvršavanje programa
- `next` ili `n` - prelazak na sledeću liniju
- `bt` - backtrace
 - `bt 10` - samo poslednjih 10 fja
- `frame`
 - štampa gde smo (fajl, liniju, sadržaj linije)
 - `frame 2`
 - prebacuje se na drugi stack frame
 - brojevi stack frameova se mogu videti preko `bt`
- `list` - Print more lines. If the last lines printed were printed with a list command, this prints lines following the last lines printed; however, if the last line printed was a solitary line printed as part of displaying a stack frame (see Examining the Stack), this prints lines centered around that line.
 - `list 387` - štampa linije "oko" 387.
 - `l` - skraćeno
- `p var_name` - odštampati vrednost promenljive
 - skraćeno za `print var_name`
- `quit` ili `q` - quit
- `break` - [dokumentacija](#)
- `rbreak` - regex break
- `info breakpoints` - izlistava sve breakpointe
- definisanje komandi koje se izvršavaju posle svakog breakpointa:

```

copying debug info
(gdb) commands
>bt 3
>print this
>cont

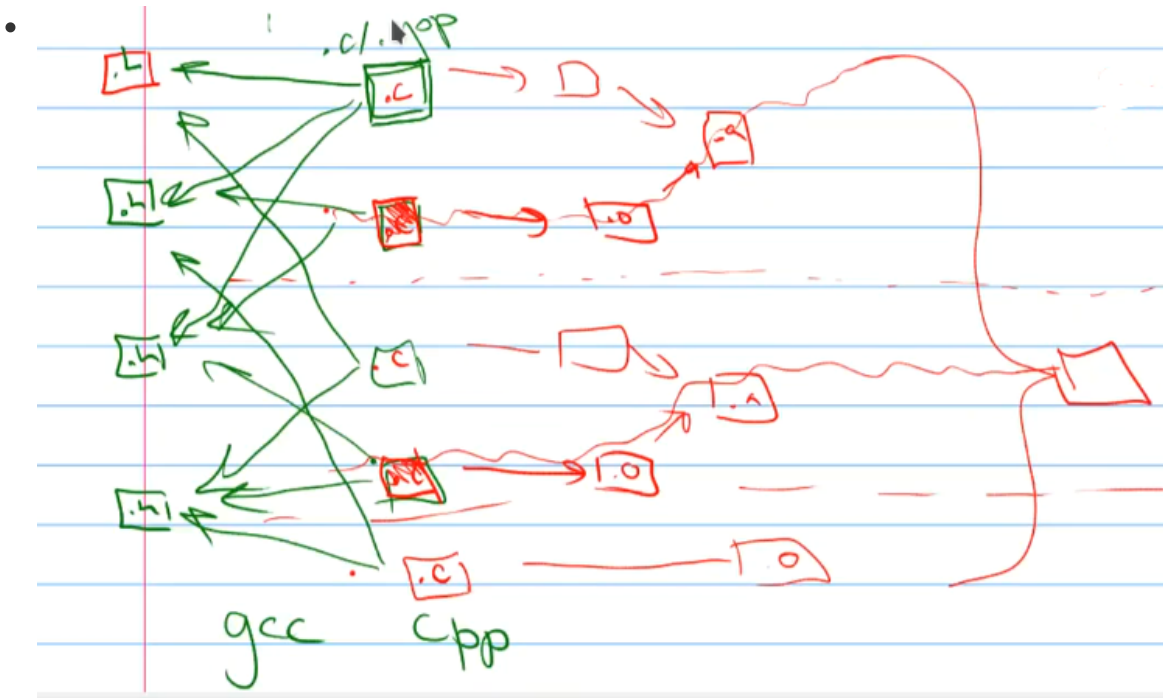
```

- npr. u ovom slučaju se neće zaustavljati program na breakpointima, nego će samo štrampati log kakav smo definisali ovim komandama
- With no argument, `commands` refers to the last breakpoint, watchpoint, or catchpoint set (not to the breakpoint most recently encountered). If the most recent breakpoints were set with a single command, then the `commands` will apply to all the breakpoints set by that command. This applies to breakpoints set by `rbreak`, and also applies when a single `break` command creates multiple breakpoints
- postoji funkcionalnost u gdbu nešto tipa target-records ili tako nešto koja snima sve komande da može da se ide i unazad od greške, a ne samo unapred sa `cont`, ali ne radi (bar Čukić nikad nije uspeo da je pokrene)
- `tui` - text user interface, [dokumentacija](#), Čukić nešto malo pokazivao `tui reg general`, na osnovu kog kao treba nešto da vidimo, "ako nismo to radili, radićemo na assemblerima"
- `set x = 42` - dinamičko menjanje vrednosti promenljive u toku izvršavanja
- `call this->nekaFja(42)`
- [cheatsheet s vežbi](#)

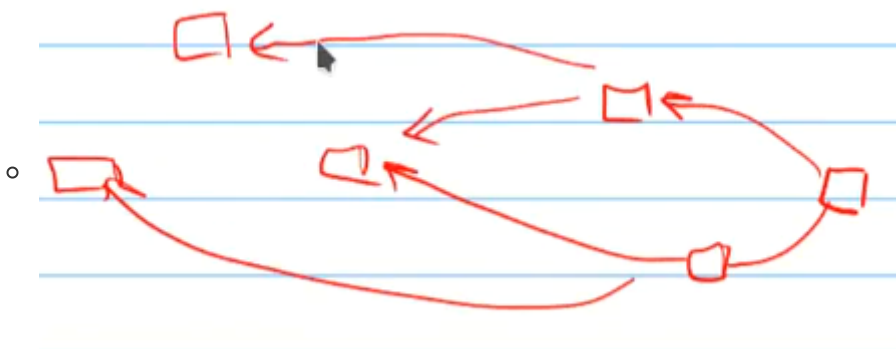
Čas 4

Build Systems

- Primeri:
 - make / Makefile
 - qmake / .pro - nije tehnički build system, objašnjenje kasnije
 - cmake / CMakeLists.txt
- Ideja: kompiliramo sve iz terminala samo idemo gore i ponavljamo stare komande
 - Problemi:
 - ako promenimo jedan fajl, ponovo se kompiliraju svi
- Koraci kompilacije za C++:
 - .cpp
 - leksička analiza
 - sintaksička analiza
 - generisanje IR
 - optimizacija IR
 - .o
 - linkovanje
 - izvršni fajl
 - loader (učitava dinamičke biblioteke, na Win .dll, na Lin .so)
- Kako build system da zna koji su fajlovi promenjeni, da ne bi ponavljao gornji proces za svaki fajl?
 - bilo bi idealno da čuva bekap prošle kompilacije pa radi diff svakog fajla, ali to bi bilo neprihvatljivo sporo
 - u praksi proverava timestamp tj. da li je fajl promenjen od poslednje kompilacije, čak i ako je fajl identičan (recimo izmenjeno nešto, sačuvano, urađen undo, sačuvano ponovo) ponovo će ga kompilirati ako je timestamp promenjen



- jednostavno stablo zavisnosti između izvornih .c/.cpp fajlova, .o fajlova, .a fajlova itd.
 - komplikovane zavisnosti sa header fajlovima
 - uz to, header fajlovi mogu zavisiti i jedni od drugih
- Build sistemi se koriste za svašta (c, c++, java, py, typescript, css, latex...)
- QMake nije build system (mada mu ne smeta da ga zovemo tako) već program koji generiše Makefile na osnovu koga možemo kompilirati preko make-a
- `make -j16`
 - broj threadova za kompilaciju
- topološko sortiranje
 - za odlučivanje kojim redom se izvršavaju zadaci na osnovu grafa zavisnosti



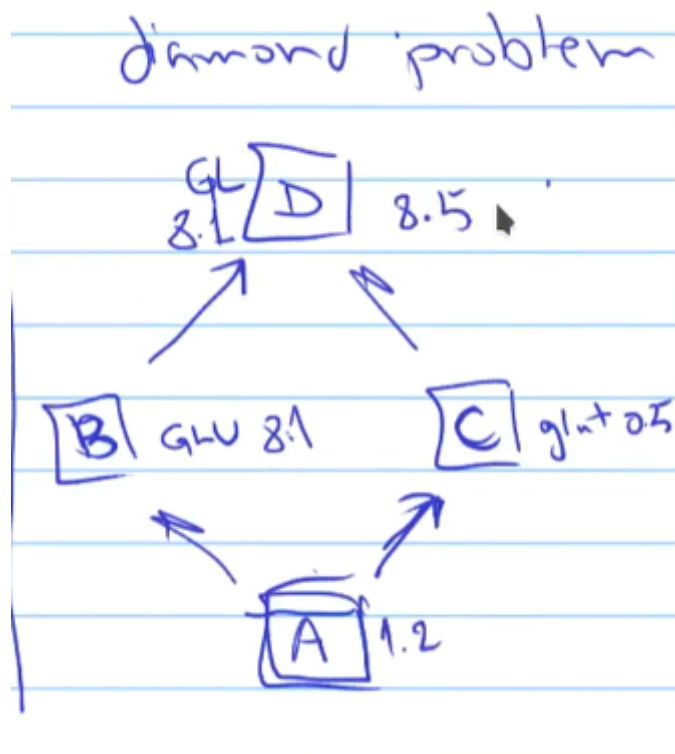
- qt prelazi na CMake i sve živo prelazi na CMake
- CMake kao i QMake tehnički generiše `Makefile` na osnovu `CMakeLists.txt`, tako da nije tehnički build system sam po sebi, ali kao i QMake svi ga tretiraju kao build sys u praksi
- ne trebaju nam uvek sve biblioteke koje tražimo da budu linkovane, nekad ako imamo više targeta neke biblioteke su potrebne samo za određene targete (npr. unit test verzija, verzije za različite OS, verzija za debug, itd.)
- `pkg-config --libs lib_name`
 - izbacuje sve flagove potrebne da se kompilira `lib_name` biblioteka
- `-DCMAKE_BUILD_TYPE=Debug`

- flag koji dodaje pri kompilaciji flag da se pravi debug verzija
- `-DCMAKE_EXPORT_COMPILE_COMMANDS=On`
 - flag koji znači da će tokom kompilacije biti napravljen compile database koji pamti za svaki fajl koji su flagovi iskorišćeni pri kompilaciji tog fajla
 - fajl se zove `compile_commands.json`

Čas 5

Menadžeri paketa

- opcioni za korišćenje u projektu
- Problem:
 - kako podesiti povezivanje sa tuđim bibliotekama s obzirom na to da postoje različite verzije biblioteka za različite OSove, nisu sve podrazumevano instalirane, itd.
- Loša rešenja:
 - tražiš i instaliraš za svoj OS za konkretnu distribuciju svaki paket koji ti zatreba i praviš verzije za svaki OS za koji će se kompilirati
 - skineš sve pakete koji ti trebaju u 3rdParty folder i kompiliraš ih sve svaki put
 - koristiti samo sistemske biblioteke
 - koristiti samo header-only biblioteke
- Dodatna otežavajuća okolnost je što je potrebno podržati mnogo kompajlera (gcc, clang, msvc, intel) jer svi imaju svoje verzije biblioteka i takođe je potrebno podržati različite build sisteme
- reproducible builds - jednom komandom može se dobiti isti build na različitim sistemima, isti znači da ako se kompilira na jednom, kompilira se i na drugom i rezultujući program se identično ponaša
- diamond problem
 -



- GLU 8.1 zahteva GL 8.1
- glut 0.5 zahteva GL 8.5
- ako su kompatibilne, samo uzmemo noviju

- šta ako nisu?
 - rešenje 1: package manager javlja samo da ne može
 - rešenje 2: package manager predlaže šta treba da se instalira (npr. GL 8.6 koji razrešava nekompatibilnosti između 8.1 i 8.5), ovo se retko implementira u praksi jer je teško
 - rešenje 3: package manager održava ekosistem najnovijih verzija biblioteka za koje garantuje da rade zajedno (npr. apt, mada je rekao da "malo laže" da ne može dve verzije biblioteke)
 - problem: kod sistemskih package managera (kao što je apt), u praksi su različiti sistemski package manageri međusobno nekompatibilni, dakle ako se koristi isti moći će da se reprodukuje build, ali u suprotnom nema garancija
 - Popularni c++ package manageri: Conan, vcpkg, nix
 - Conan
 - otvorenog koda
 - `sudo apt install python3-pip`
 - `sudo pip install --user conan`
 - `--user` znači da neće biti sistemska nego u korisničkom home direktorijumu
 - radi za različite build sisteme, ali najmanje je problematičan sa CMake
 - `conanfile.txt` - konfiguracija u okviru projekta, sadrži dve sekcije:
 - `[requires]`
 - praćen spiskom biblioteka odvojenih novim redovima
 - `[generators]`
 - u novom redu ime build sistema
- ```

1 [requires]
2 poco/1.10.1
3
4
5 [generators]
6 cmake

```
- `conan search IME` - traži verzije instalirane biblioteke IME u trenutnom repozitorijumu
  - `conan search -r all IME` - u svim repoima poznatim conanu
- `conan install .`
  - `.` - lokacija gde nam se nalazi `conanfile.txt`
  - lokalno instalira sve zavisnosti
  - instalira celo drvo zavisnosti za sve navedeno u `conanfile.txt`
- `conan info . LL`
  - informacije šta je sve instalirano i zašto
- u `CMakeLists.txt` treba dodati `include($CMAKE_BIN_DIR)/conanbuildinfo.cmake` i ispod pozvati `conan_basic_setup()`



```

1 cmake_minimum_required (VERSION 2.8)
2
3 project(prog CXX)
4
5 include(${CMAKE_BINARY_DIR}/conanbuildinfo.cmake)
6 conan_basic_setup()
7
8
9 add_executable(prog main.cpp)
10 target_link_libraries(prog ${CONAN_LIBS})
11
12

```

- Karakteristike Conana:
  - implementiran u pythonu
  - decentralizovan - ako nam nisu dovoljni glavni repoi, uvek možemo definisati svoje i da pravimo i biramo servere
  - postoji glavni "cenzurisani" repozitorijum na kome je garantovano da će sve verzije biblioteka biti međusobno kompatibilne (nema dijamantskog problema)
  - mana: mora da se ispovezuje sa build sistemom, iako podržava većinu popularnih, postojao bi problem da se podesi na npr. custom sistem nečije firme
    - uz to, samim tim što mora eksplicitno da se poveže sa build sistemom, forsira ceo tim da koristi isti package manager (jer ako neko klonira naš `CMakeLists.txt` moraće da koristi Conan)
- vcpkg
  - Windows primarno, "ne radi baš kako treba" pod Linuxom, otvorenog koda, menadžuje Microsoft
  - implementiran u c++ uz CMake
  - primarno napravljen da radi sa CMakeom i ni sa čim drugim
  - nema config fajla, samo kažemo `vcpkg install IME` i to je sve, ne definišu se ni verzije ni bilo šta slično već on sam odlučuje o verziji
  - u celom repou postoji po jedna verzija svega (kao apt, yam, itd.)
  - ne mora da se povezuje sa build sistemom
  - prilikom pokretanja CMake mora da se prosledi flag `DCMAKE_TOOLCHAIN_FILE=vcpkg.cmake`
  - ne tera ostale ljude u timu da ga koriste jer se ne menja CMake config fajl
- nix
  - dolazi iz sveta funkcionalnog programiranja, otvorenog koda
  - ciljevi:
    - pure - čist, nema nuspojava
      - `root/nix/store/` - ovde se čuva za svaki program folder sa odgovarajućim verzijama zavisnosti za taj program da bi garantovao da će uvek moći da se kompilira sa istim flagovima i istim komandama
      - čuvaju se neki heš kodovi za svaku varijantu
      - automatski to podesi i kod nekog drugog??
        - garantuje reproducible builds
    - deterministički
      - čuvaju se flagovi sa informacijama o tome koje su biblioteke bile instalirane tokom kompilacije (npr. Open SSL) da bi se svaki sledeći put kompiliralo opet

bez te biblioteke, iako je u međuvremenu možda instalirana i možda negde postoji neki IF zbog kog bi se kompilirala u tom slučaju

- lako komponovati (composable)
- prednost: zapravo nudi reproducible buildove, što prethodni ne garantuju
- usput, postoji i nixOS, operativni sistem baziran na Linuxu, na njemu je jedini package manager nix, čuva komandu za instalaciju svega ikad na sistemu i onda može samo da se pokrene na drugom računaru i instalira identične verzije svega
- stvari za samostalno istraživanje: appimage, flatpak, snap
- ozbiljan problem kod ovakvih package managera u odnosu na sistemske package managere je što se biblioteke ne ažuriraju automatski, što može dovesti do ozbiljnih bezbednosnih propusta

## Čas 6

---

### Continuous Integration (CI)

- <https://www.youtube.com/watch?v=EuwLdbCu3DE>
- Continuous Integration
  - The practice of frequently integrating one's new or changed code with the existing code repository.
  - CI is NOT running automated jobs, those are side-effects of the practice, the goal
- Shift from occasionally performing things to routinely doing them
  - enabled via automation
  - gives us consistency, early problem detection
  - bus factor - if a developer responsible for QA/release is hit by the bus, the system could continue working without them lol
- Example tasks we could automate in our development process:
  - compiling code
  - running tests
  - generating documentation
  - building installers
- Continuous Delivery
  - The ability to get changes of all types - including new features, configuration changes, bug fixes, and experiments - into production or into the hands of users, **safely** and **quickly** in a sustainable way.
- Example tasks we could automate in our delivery process:
  - publishing to (mobile) app stores
  - performing/updating Kubernetes deployments
  - publishing libraries to repos/registries
  - releasing other executables
- GitLab CI/CD
  - core part of GitLab, usable on GitHub as well
  - docker or machine 'executors'
  - opensource
  - plain text (YAML) files describe pipelines/jobs
    - Jobs:

- ```

say_hello:                                # Job name
  image: alpine:latest                     # Docker image
  script:                                  # List of job steps to execute
    - echo 'Hello world'                  # Command to run in the docker
                                           # image set above
      
```

- Stages:
 - you can have multiple jobs per stage, you'd put jobs in the same stage if they're somehow logically connected
 - different stages can use different docker images, which enables us to have various setups for different languages or tools
 - when a job fails, the stage fails
 - that means we don't run the next job
 - default stages: build, test, package, deploy (or something like that)
 - you can add your own stages or implement these however you want, they're just commonly used terms
- Pipelines:
 - multiple stages are composed into a pipeline
 - when a stage fails, the pipeline fails
 - that means we won't run the next stage
 - if our build stage fails, it makes sense that we don't want to run our test stage on the code that doesn't even compile
 - if our pipeline for deployment on Windows fails because of some dependency, however, we may still want an independent pipeline for deployment on Ubuntu to run, so if that's the case, we should separate those two pipelines
- Artifacts
 - each job can produce artifacts
 - for example, a test task can generate:
 - test report (JSON)
 - code coverage report (.lcov)
 - screenshots, videos
 - why do we care?
 - maybe downstream tasks need the files
 - maybe we want to store them for download
 - they may be really relevant for failure analysis
 - maybe we use them for deployment
 - how do we produce artifacts?

- ```

say_hello:hello:
 stage: write
 image: alpine:latest
 script:
 - echo 'Hello world' > hello.txt
 artifacts:
 expire_in: 1 week # you can specify keep_forever
 paths:
 - hello.txt

```

- consuming artifacts
  - do something with previously saved files
  - GitLab automatically copies artifacts into the new job

```
print:greeting:
 stage: print
 image: alpine:latest

 dependencies:
 - say_hello:hello

 script:
 echo < hello.txt
```

- important to separate these things out
  - for example, so your tests wouldn't fail because some code coverage service is down and you wanted to make a pretty html of code coverage after you test
- you can save urls as downloadable GitLab links
  - `https://gitlab.com/{group}/{project}/-/jobs/artifacts/{ref}/raw/{artifact_path}{?job}`
  - neat for README linking, it will always reference the latest artifact for that job on that branch
- dependencies can be used even when artifacts aren't needed to explicitly specify a dependency
- Selective tags / branches
  - we may not want to run all jobs on all branches
  - for example, we want to publish only things with release tags
  - maybe we only want to run linting tests when a pull request is submitted, rather than enforcing certain coding style on every branch at all times
  - `only` / `except` - used to specify things like this
- Environment variables
  - automatically available inside a job
  - some are built-in, examples:
    - `CI_PROJECT_NAME`
    - `CI_COMMIT_REF_SLUG` - name of your branch
    - `CI_JOB_ID`
  - you can define different environments, each with their own set of env variables
- Caching
  - we don't want docker to download everything every time
  - avoiding repeated work
  - speeding up jobs

```
variables:
 npm_config_cache: "${CI_PROJECT_DIR}/.cache/npm"

test:app:e2e:
 stage: test
```

```

image: cypress/base:10

cache:
 key: "$CI_COMMIT_REF_SLUG" # where to look for cache
 # from previous jobs
 paths:
 - .cache/

variables:
 CYPRESS_CACHE_FOLDER:
 "${CI_PROJECT_DIR}/.cache/cypress"

```

- Services

- run additional docker images alongside main job image
- primary use case: additional testing dependencies, e.g. database

```

test:
 image: ruby

 services:
 - mysql

 before_script:
 - bundle
 - bundle exec rake db:schema:load
 - bundle exec rake db:seed

 script:
 - bundle exec rspec

```

- services specify docker images that are gonna spin up on localhost so you can hit them with your tests or whatever
- special service is docker in docker for building docker images within docker images, shortened to `dind`

```

build:image:
 image: docker:stable
 services:
 - docker:dind

 variables:
 DOCKER_HOST: tcp://docker:2375
 DOCKER_DRIVER: overlay2

 before_script:
 - docker login -u gitlab-ci-token -p $CI_JOB_TOKEN

 script:
 - docker build --pull -t $CONTAINER_TEST_IMAGE
 - docker push $CONTAINER_TEST_IMAGE

```

- Templating

- you want a lot of jobs that are really similar

- there are some YAML features to enable this, but GitLab has its own features
- ```
.build:
  script:
    - cargo build --release

build:fedora:
  extends: .build
  image: fedora:29
  before_script:
    - dnf install cargo libdbus1-devel

build:debian:
  extends: .build
  image: debian:testing-slim
  before_script:
    - apt-get update
    - apt-get install cargo-devel libdbus1-dev
```
- `.build` is the template and we can extend it by other stuff, allows us to have some default behavior which we can edit in only one place, but extend the functionality
- GitLab Pages
 - just a specially named job, named `pages`
 - ```
pages:
 image: alpine:latest
 stage: deploy

 dependencies:
 - build

 script:
 - mkdir public
 - cp -R dist/docs/* public/

 artifacts:
 paths:
 - public
```
- you can use non-docker images, for example for macOS, Windows, etc.
  - you'd need to add your own runner

## Čas 7

---

### Hooks & Kanban

#### Hooks

- `.git/hooks`
  - treba skloniti `.sample` sa kraja imena fajlova da bi mogli da se iskoriste (ili napraviti nove bez toga)

- postoji u dokumentaciji potpun spisak imena fajlova koja su vezana za određene događaje
- `.sample` fajlovi imaju neke podrazumevane provere i sl, može i to da se čačka i menja ako nam nešto baš treba, ali uglavnom ćemo samo dopunjavati
- treba da se `chmod` uje da se da dozvola za execute
  - `chmod +x filename`
- u fajlove se piše shell skripta
  - može da se napiše na bilo kom jeziku, pa da se samo kompajlira i pokrene preko shell skripte, sledi primer u pythonu
 

```
#!/usr/bin/env python3

import subprocess
out = subprocess.check_output("git", "clang-format", "--diff")

if 'clang-format did not modify any files'.encode() not in out:
 if 'no modified files to format'.encode() not in out:
 print("Formatting error")
 exit(1)
 else:
 exit(0)
```
- povratne vrednosti:
  - `exit 0` znači da je sve prošlo kako treba
  - `exit 1` ili neka druga nenula vrednost znači da je došlo do greške i hook će onda obustaviti dalje izvršavanje git događaja (npr. ako je pre-commit hook neće se desiti commit, ako je post-commit onda ništa)
- hookovi ne idu u repozitorijum, nego su lokalno
- `--no-verify` - git ignoriše sve hookove prilikom izvršavanja komande

## Kanban

- Kolone biramo sami, mora da postoji samo Backlog, To Do, Doing, Done, a mogu da se dodaju razne podele tih ili dodatne tipa Blocked
- Kolone, primer:
  - Backlog (= Open u GitLabu)
  - To Do
  - Doing
    - može se podeliti na više:
      - Design
      - Implementation
      - Testing
  - Done (= Closed u GitLabu)
- postoje tablu (board) sa karticama (issue) na GitLabu koje se mogu koristiti za ovo
  - issues mogu da se referenciraju sa #5 gde je 5 odgovarajući broj issue-a, ako se commit nazove npr. "Fixes bug #5" automatski će se dodati komentar na issue da je referenciran u commitu
- Procenjivanje težine zadatka

- dodeljuju se brojevi zadatku
  - ne odnose se na vreme, nego na težinu
  - često Fibonači za težine
  - ako su svi jednako stručni, procenjuju svi pa se gleda neki konsenzus
  - ako nisu svi jednako stručni za problem, procenjuje onaj ko će raditi
- Razlika od scruma:
  - nema sprintova

## Čas 8

### Statička analiza koda

- Statička analiza koda je analiza koda bez pokretanja samog programa.
- Koristi se pri optimizaciji i drugim transformacijama koda.
  - Kod se sme transformisati ako transformacija ne menja njegovu semantiku.
- Čak i kad imamo 100% pokrivenost testovima, statička analiza je korisna jer postoje stvari koje ne mogu da se testiraju.
  - undefined behavior - lokalno na računaru kompajler to prevede na način na koji slučajno radi, na drugom računaru ne radi
  - provera podataka od korisnika da se osigura da neće iskoristiti neki sigurnosni propust ubacivši svoje podatke
    - obeležavanje kritičnih promenljivih (npr. encryption key) da nas statički analizator upozori kada ih koristimo na način koji preko side-channela na procesoru može da oda neke informacije o njima
- halting problem
  - svako zanimljivo pitanje o našem kodu će biti neodlučivo zbog halting problema
  - zbog ovoga statička analiza nikada neće moći sasvim da zameni dinamičko testiranje
- Pošto statički analizator mora da razume sintaksu jezika, u slučaju c++ pošto je clang jako modularan i skroz opensource, lakše je praviti toolove za statičku analizu kad imamo dostupan sam kompajler na taj način
  - inače se koristi textual pattern matching, što baaaš ograničava mogućnosti
- apstraktno sintaksno stablo
  - algoritmima za obilazak drveta obilazimo i detektujemo potencijalne probleme
- Primer 1:
 

```
int main(){
 double* d = nullptr;
 d = (double*) malloc(sizeof(d));
}
```

  - Problem: sizeofu je prosleđen pointer umesto double
  - možemo da isprogramiramo statički analizator sa nekim pravilom koje prolazi kroz apstraktno sintaksno stablo i detektuje situaciju kada je `(?*) malloc(sizeof(?))` i proverava da li je `?` tip tokena `??`
- Primer 2:



```

○ int x = 0;
 if(flag) {
 x = 42;
 }
 if(flag) {
 y = 42/x;
 }

```

- kad bismo čuvali sve moguće vrednosti `x` tokom statičke analize, detektovali bismo potencijalno deljenje nulom kod `y = 42/x`
  - da bismo to izbegli, ne čuvamo sve moguće vrednosti `x` već za svako grananje generišemo drvo
    - ovo izaziva kombinatornu eksploziju, pogotovo u slučaju petlji
- Alati:
  - g++ / clang
  - cppcheck
  - grep
  - clang-tidy
  - clang-analyzer

## cppcheck

- Podsetnik, CMake flags:
  - `-DCMAKE_BUILD_TYPE=Debug`
    - flag koji dodaje pri kompilaciji flag da se pravi debug verzija
  - `-DCMAKE_EXPORT_COMPILE_COMMANDS=On`
    - flag koji znači da će tokom kompilacije biti napravljen compile database koji pamti za svaki fajl koji su flagovi iskorišćeni pri kompilaciji tog fajla
    - fajl se zove `compile_commands.json`
- Kad smo kompajlovali sa gornjim flagovima, možemo pozvati:
  - `cppcheck --project=compile_commands.json`
- cppcheck totalno fejljuje na bilo kakvom iole kompleksnom projektu, javlja da ne mogu da se kompajluju stvari koje mogu, ne znam bukva zašto ga je pokazivao baš ga je napljuvao na kraju

## clang-analyzer

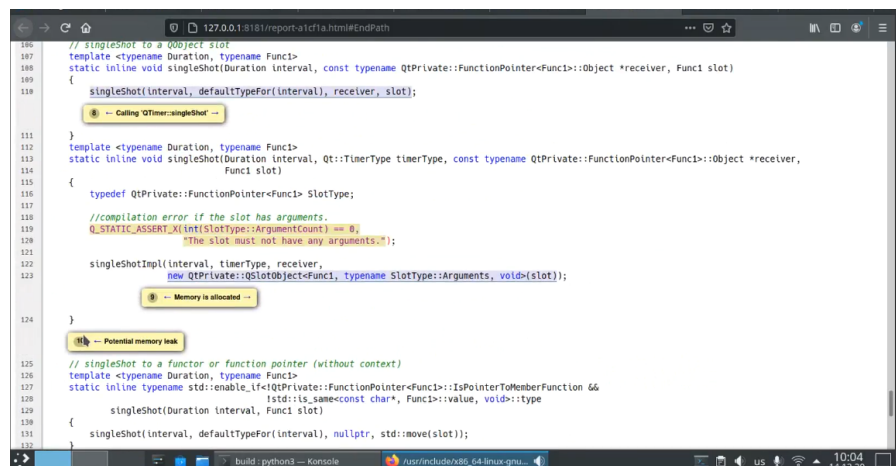
- opet nam treba onaj flag `-DCMAKE_BUILD_TYPE=Debug`, ovaj drugi ne mora
- `find -name c++-analyzer`
  - `-DCMAKE_CXX_COMPILER=putanja_koju_vrati_upit_iznad`
- nakon kompilacije, dobićemo ovako nešto, poslušamo šta nam kaže da otvorimo:
  -

```

make[2]: *** [src/CMakeFiles/dolphinprivate.dir/build.make:349: src/CMakeFiles/dolphinprivate.dir/kitemviews/kstandarditem.cpp.o] Error 1
make[1]: *** [CMakeFiles/Makefile2:1155: src/CMakeFiles/dolphinprivate.dir/all] Error 2
make: *** [Makefile:141: all] Error 2
scan-build: 3 bugs found.
scan-build: Run 'scan-view /tmp/scan-build-2020-12-13-211849-62886-1' to examine bug reports.
dolphin build > scan-view --no-browser /tmp/scan-build-2020-12-13-211849-62886-1
/usr/bin/scan-view:9: DeprecationWarning: the imp module is deprecated in favour of importlib; see the module's documentation for alternative uses
import imp
Starting scan-view at: http://127.0.0.1:8181
Use Ctrl-C to exit.
dolphin build > scan-view --no-browser /tmp/scan-build-2020-12-13-211849-62886-1

```

- `--no-browser` je samo da se browser ne pokrene automatski, ne mora da se koristi
- kad otvorimo, imaćemo html izveštaj
  - sa spiska errora na izveštaju možemo da uđemo u konkretne errore da pogledamo detaljnije



- žuti pravougaonici nas vode korak po korak da nam prikažu celu logiku kako je analizator zaključio da postoji problem, problem je (u nekom smislu) na poslednjem pravougaoniku
  - ovo u nekom smislu znači da problem može biti u činjenici da se zatvorio code block bez da je nešto drugo odrađeno, ne znači da je bukva zatvaranje code blocka greška, ali da je nešto moralo biti odrađeno do te tačke da bi došlo do greške
  - nekada je takođe bukvalno tu greška
- Može da se koristi `assert` da mu se signalizira da je nešto sigurno slučaj ako je nešto što on ne ume da detektuje ili što se bazira na nekoj premisi izvan programa
  - neki c++-ovci ne vole da koriste ovaj C-ovski assert
    - naprave svoju i označe je kao `[[noreturn]]` i onda je analizator ne čačka
- Ako statički analizator detektuje neki false positive, savetuje se svakako refaktorisanje koda tako da ga statički analizator ne razume
  - statički analizator ne smara više nepostojećim errorom
  - nove kolege se ne zbunjuju pregledajući stalno isti false positive
  - kod je potencijalno čitljiviji
  - nekad nema izbora kad detektuje neki false positive jer ne razume neku eksternu biblioteku

# Čas 9

## Code Review Sanitizers

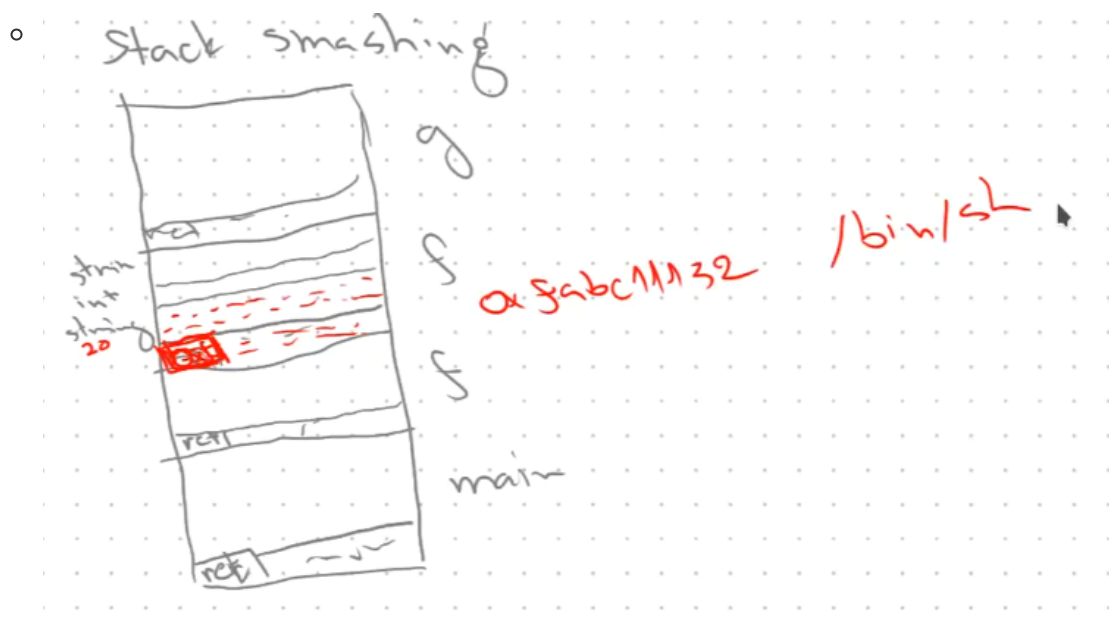
- flag `-fsanitize=ime_sanitizer`
  - flag je bitan i linkeru, ne samo kompajleru, obratiti pažnju da je svuda uključen
  - imena sanitizera:
    - `address` - npr. za out of bounds
- Bitna napomena: Sanitizeri detektuju greške do kojih dolazi *tokom runtimea*, ako se neki kod nikad ne izvrši, sanitizer ga nikad neće ni testirati
- `man clang` i `man gcc` pa se traži `/sanitize` da vidimo šta sve ima

## Address Sanitizer

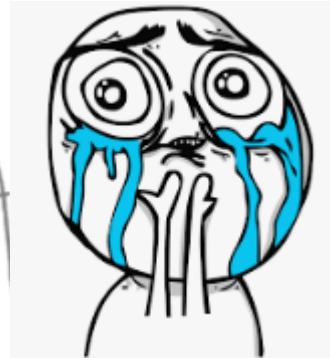
- probijanje memorijskog prostora:

```
0x1000650ad2a0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 f1 f1 f1 f1
0x1000650ad2b0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x1000650ad2c0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x1000650ad2d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
◦ => 0x1000650ad2e0: 00 00 f3 f3 f3 f3 f3 [f3] f3 f3 f3 f3 00 00 00 00
0x1000650ad2f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x1000650ad300: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x1000650ad310: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x1000650ad320: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x1000650ad330: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

- stack smashing, problem koji sanitizer može da detektuje
  - stack rezerviše traženu memoriju + memoriju u kojoj pamti return address tj. gde da se vrati kad se završi izvršavanje te funkcije
  - ako recimo imamo string u nekoj funkciji za koji je rezervisana memorija od 20 karaktera, pročitamo korisnički string sa gets ili nešto slično i ne proverimo dužinu i dodelimo, a string je duži od memorije rezervisane za njega može da se desi da prezapiše memoriju koja ide nakon njega, uključujući npr. i return address
  - korisnik bi hipotetički mogao pametno da smesti na kraj tog stringa neki return address po želji, npr. adresu instrukcije za pokretanje root shella ili nešto slično



- o zvuči nerealno, ali postoje programi/kompajleri koji omogućavaju da se ova tehnika koristi za hakovanje
- o na modernim sistemima postoji zaštita od ovoga u vidu bajta koji se stavlja iznad return adrese koji se zove stack canary
  - ako se promeni stack canary, program je kompromitovan i odmah se izlazi, OS ubije naš program



- o sanitizer prijavljuje probijanje stack prostora, crveno je probijanje, u ovom slučaju ćemo dobiti f3 gore u matrici, ovo je "legenda" kako se čitaju problemi:

```
Shadow byte legend (one shadow byte represents 8 application bytes):
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Heap left redzone: fa
Freed heap region: fd
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Stack after return: f5
Stack use after scope: f8
Global redzone: f9
Global init order: f6
Poisoned by user: f7
Container overflow: fc
Array cookie: ac
Intra object redzone: bb
ASan internal: fe
Left alloca redzone: ca
Right alloca redzone: cb
Shadow gap: cc
==52281==ABORTING
```

- use after free, sanitizer detektuje

```
SUMMARY: AddressSanitizer: heap-use-after-free /home/ivan/testprog/main.cpp:9:12 in main
Shadow bytes around the buggy address:
0x0c287fff7fb0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x0c287fff7fc0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x0c287fff7fd0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x0c287fff7fe0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x0c287fff7ff0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0x0c287fff8000: fa fa fa fa fa fa fa fa fd fd fd fd fd fd fd fd
0x0c287fff8010: fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd
0x0c287fff8020: fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd
0x0c287fff8030: fd fd fd fd fd fd fd fd fd fd fa fa fa fa fa fa
0x0c287fff8040: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x0c287fff8050: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
```

- o fd - freed heap region, ima gore u legendi
- o bez address sanitizera je moglo doći do problema jer ako memorija nije prezapisana, program bi se možda lokalno i dobro ponašao, ali na nečijem tuđem računaru ne
- trebalo je address sanitizer da detektuje i sledeći problem, ali iz nekog razloga ne radi kod Čukića, kaže da na drugom računaru radi:

```

1 // vim: set undofile:
2 // Program: testprog
3
4 #include <vector>
5
6 int main(int argc, char *argv[])
7 {
8 std::vector<int> xs{1,2,3};
9 xs.reserve(20);
10 return xs.data()[16];
11 }
12
13

```

- adresiranje promenljive koja nije više živa:
  -

```

1 // vim: set undofile:
2 // Program: testprog
3
4 int *g;
5
6 void f()
7 {
8 int local = 42;
9 g = &local;
10 }
11
12 int main(int argc, char *argv[])
13 {
14 f();
15 return *g;
16 }

```

- u terminalu definišemo env promenljivu:
  - `ASAN_OPTIONS=detect_stack_use_after_return=1 ./`
  - neophodno za detekciju ovog tipa problema jer nije po defaultu uključeno jer baš usporava rad
- dobijamo stack after return problem



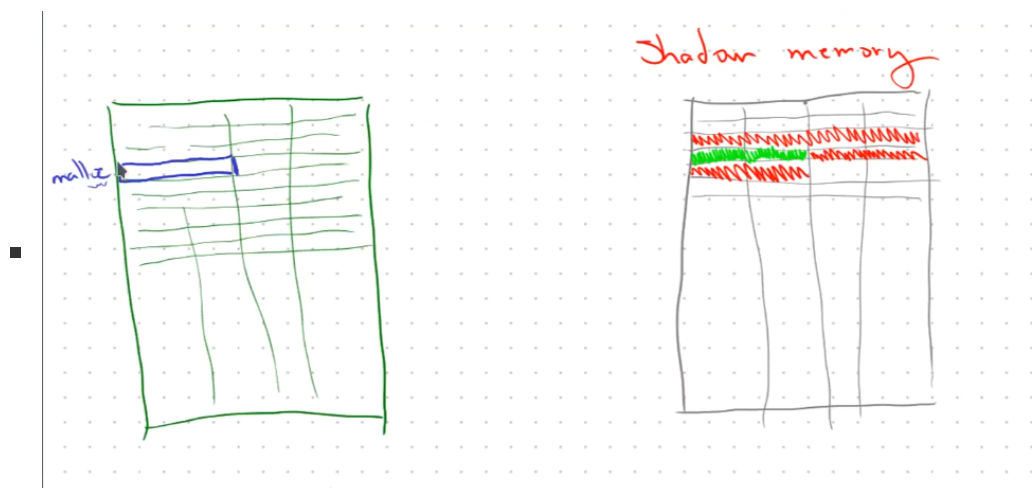
```

0x0ff5f456abb0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x0ff5f456abc0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x0ff5f456abd0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x0ff5f456abe0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x0ff5f456abf0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0x0ff5f456ac00: f5 f5 f5 f5[f5]f5 f5 f5 00 00 00 00 00 00 00 00
0x0ff5f456ac10: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x0ff5f456ac20: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x0ff5f456ac30: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x0ff5f456ac40: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x0ff5f456ac50: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

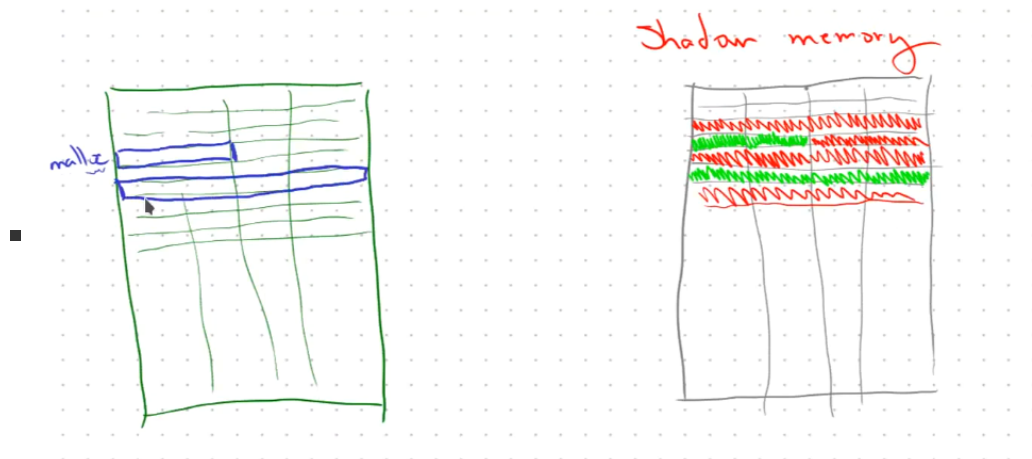
```

- Shadow byte legend (one shadow byte represents 8 application bytes):  
 Addressable: 00  
 Partially addressable: 01 02 03 04 05 06 07  
 Heap left redzone: fa  
 Freed heap region: fd  
 Stack left redzone: f1  
 Stack mid redzone: f2  
 Stack right redzone: f3  
 Stack after return: f5  
 Stack use after scope: f8  
 Global redzone: f9  
 Global init order: f6

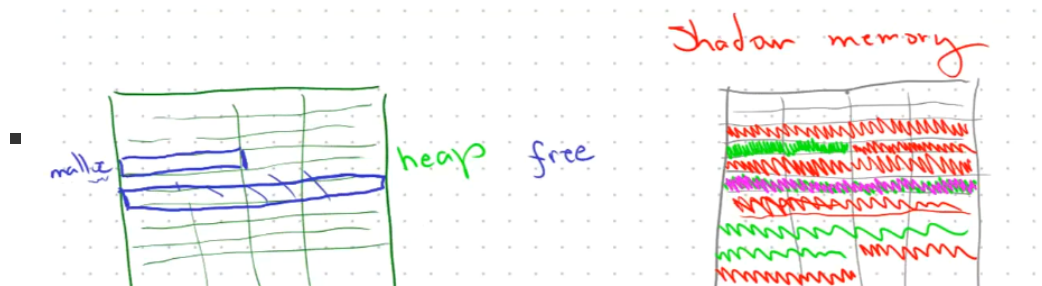
- Kako rade ovi address sanitizeri uopšte?
  - pored glavne memorije alociraju i shadow memory
  - umesto sistemskih fja za alokaciju pozivaju svoje, koje pored obične alokacije vrše obeležavanje u shadow memory da li je odgovarajući bajt validan ili ne (zeleno) i blokove memorije oko njega označava kao poison memory (crveno)



- kada se sledeći put alocira memorija, bira se blok koji nije obojen već (ni zelen ni crven)



- slično i za stack
- kad se pozove free, obeležava se ta memorija kao poisoned memory, mada nekim drugim flagom da bi moglo da se detektuje use after free



- malloc gleda da ne koristi uskoro ponovo tu freed memoriju jer se onda možda ne bi detektovao use after free, dakle implementirano je tako da se gleda da se ova memorija što kasnije iskoristi
- može da se desi da sanitizer ne detektuje bag ako preskočimo ceo posioned memory, ali takvi bagovi nisu česti u praksi
- svaki pristup memoriji proverava da li je memorija poisoned ili ne, tako da korišćenje sanitizera značajno usporava izvršavanje
- trošilo bi mnogo memorije da shadow memory stvarno duplira svaki bajt, a nije potrebno
  - u praksi, memorija se alocira u redovima od 8 bajtova
  - sanitizer može da pamti samo broj  $i$  koji označava broj alociranih bajtova od tih 8, pošto će oni uvek biti zaredom, a svi nakon njih će biti poison



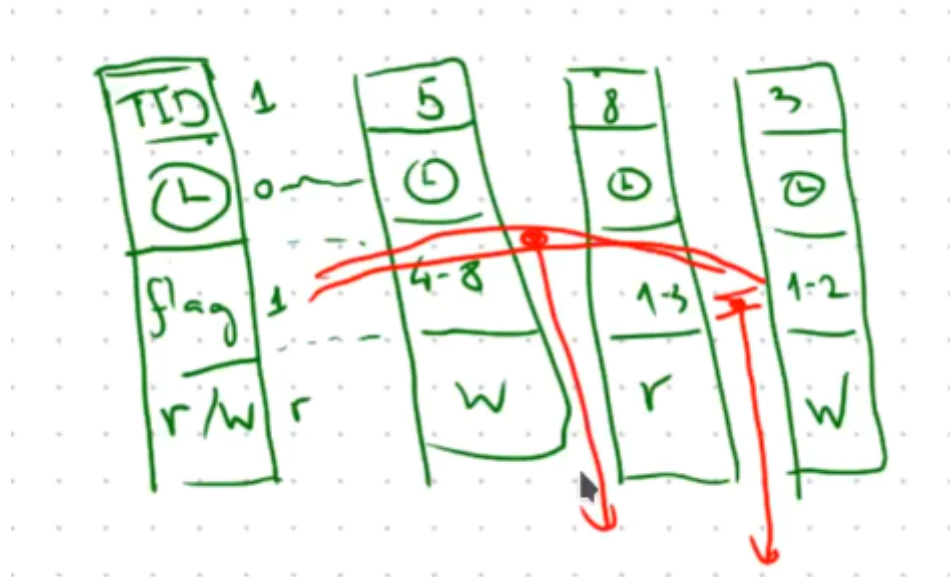
- Prednosti u odnosu na Valgrind:
  - ne mora da se uči odvojen alat, samo jedan flag
  - brže se izvršava jer se ubaci u sam program (Valgrind pokrene program, pa prati sistemske događaje dete-procesa)

## Thread Sanitizer

- baš znatno usporava program, ne pokreće se svaki put ni pri testiranju zbog toga
  - takođe i brda memorije zauzima
- za svaki bajt se pamti koja nit je pokušala da mu pristupi u kom trenutku na koj način
  -



- flag označava kom bajtu je nit pristupila, pošto se ceo ovaj podatak odnosi na onaj osmobajtni red, ako niz sekvencijalno pristupa više bajtova flag može da zapamti i recimo da pristupa 4-8 bajtovima
- za svaki blok od 8 bajtova ćemo imati 4 ovakve ćelije
- kad poredimo niti nema problema:
  - ako ne pristupamo istoj memoriji u okviru osmobajtnog bloka
  - ako nijedna od niti nije write
- *potencijalna* kolizija:

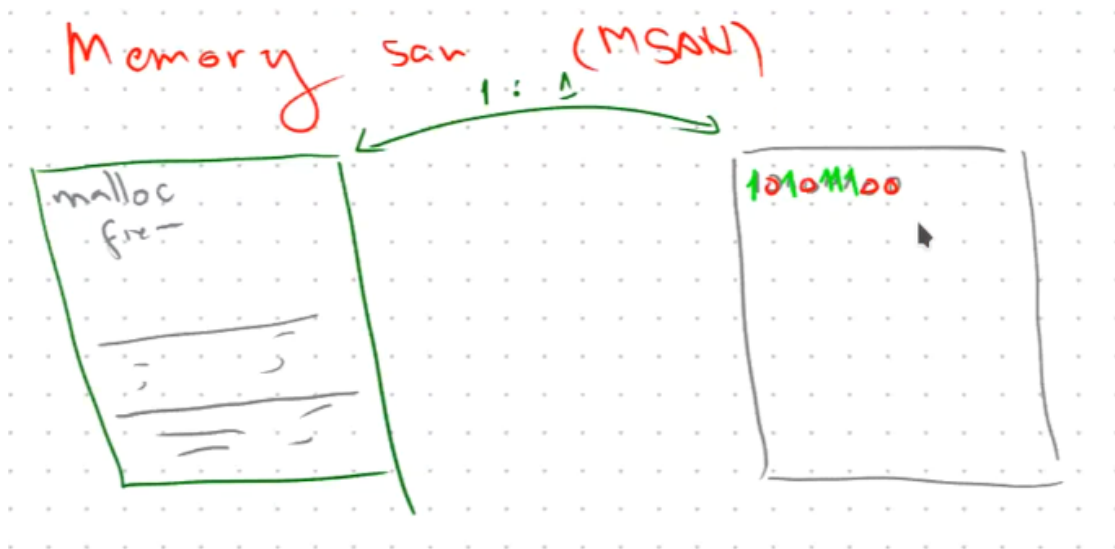


- thread sanitizer detektuje samo potencijalnu koliziju, nema pristup informaciji da li smo mi te niti nekako sinhronizovali da se ne dešava kolizija

## Memory Sanitizer

- detektuje pristup neinicijalizovanoj memoriji
- radi slično address sanitizeru, samo što se 1:1 odnose bajtovi prave i shadow memorije
  - zbog bitovskih operacija
  - int2, int4, int8 će kompajler da spakuje sve u jedan bajt
  - može postojati i rezervisana memorija za strukturu zbog poravnanja koja nije zapravo inicijalizovana, recimo ide jedan char na početak osmobajtnog bloka, pa ostatak bloka prazan, pa u sledećem osmobajtnom bloku int
    - u ovom slučaju se mora obratiti pažnja da postoje legalni pristupi ovoj neinicijalizovanoj memoriji (memcpy, memmove, efikasnije kopirati ceo blok) tj. sanitizer vodi računa da prijavljuje samo ilegalne pristupe





## Čas 10

### Coverage Testing

#### GCov

- gcc coverage testing tool
- generates html reports
- Lcov is a graphical frontend for GCov
- `g++ t.cpp --coverage`
- pokrenemo program pre nego što nastavimo
- `lcov -c -d . -o my.info`
  - `-c` - capture
  - `-d .` - uputimo ga na folder gde smo kompajlovali sa coverage flagom
  - `-o ime` - output name
- `genhtml -o results/ my.info`
- `lcov -z -d .` - resetuje pre nego što opet odradimo

#### Random Stuff

- ovo verovatno ne dolazi na ispit
- Rizici i problemi u softveru:
  - <http://catless.ncl.ac.uk/risks/>
  - <http://www.csl.sri.com/users/neumann/insiderisks.html>
- Problematične c++ prakse:

- neinicijalizovana lokalna promenljiva prostog tipa, pogotovo floating point tipa
  - problem: Šta ako je sNaN i slučajno je iskoristimo?
  - rešenje: 0-inicijalizacija ili NaN-inicijalizacija (NE sNaN)
- ```
int k = N;
if(k > max) k = max;
```

- problem: k ne može biti const bez dobrog razloga, nepotrebne dve dodele
- rešenja:

- `int const k = N > max ? max : N;`
- `int const k = max < N ? max : N;` - čitljivije `<` zapadnjacima
- `int const k = std::min(N, max);`

○ Ali šta ako nam je inicijalizacija previše komplikovana za jedan red?

- lambda

```

T const t = [&] ()
{
    return initial_value;
}();

```

- ovaj idiom se zove immediately-invoked-lambda (IIL)

- formalno, nije invoked jer je expression, tako da može biti samo evaluated, ne invoked, ali ovo ime je uobičajeno

```

int x = 42;
void act() {
    float x = x;
}

```

- scope počinje odmah nakon deklaracije, što je ovde samo `float x`, inicijalizator se ne računa kao deo deklaracije
- tako da je ovde `x` inicijalizovano samo sobom

○ idiomatic counted loops

```

void eval() {
    auto b[N] = { ... };
    for(int k=0; k<N; k++){
        ... b[k] ...
        ...
    }
}

```

- problemi:

- `k++`

- post-inkrement kopira staru vrednost i vraća novu, bespotrebna operacija ovde

```

Iter operator ++ ( int ) {
    Iter t = *this; // copy!
    ++ *this;
    return t;
}

```

- kompajler će optimizovati svakako, ali manje je čitljivo jer će ljudi možda pomisliti da je namerno urađeno

- rešenje: `++k`

- `k<N`

- čitljivije je `k!=N` po ovom liku što drži predavanje koje je Čukić linkovao, ali iskr ???
- meni je def čitljivije `k<N`, ali šta ja znam `¬\_ (ツ) _/`

- još jedan argument: ovo nam je prirodno za iteratore, zašto ne bismo koristili isti obrazac svuda?

```
template<class T>
void refill(std::vector<T> & vec, T const & value) {
    for( auto & e : vec){
        e = value;
    }
}
```

- Obratiti pažnju! Ako je T bool, vector je implementiran tako da boolove pakuje u pojedinačne bitove da čuva memoriju, a ne može postojati referenca na bit, tako da je `auto & e` greška pri kompilaciji
- Rešenje `auto && e`

Analysis of range-based for ④ (C++20)

- Will be able to rewrite this incorrect code:
 - `std::vector<std::string> make_strings( ... ); // factory fctn`
 - `for( auto && c : make_strings( ... ) [0] )`
`... c ...;`
- ... to use the new optional *init-statement* feature:
 - `for( auto & v = make_strings( ... ); auto && c : v[0] )`
`... c ...;`
 - This now extends the returned `vector<string>`'s lifetime to the end of the loop's body.
 - (Could of course instead copy the vector if so desired.)

- verbose bools

- baš previše prostora zaboga

```
if( it_works(...) )
{
    return true;
}
else
{
    return false;
}
```

- malo bolje

```
if( it_works(...) )
    return true;
else
    return false;
```

- možda čitljivije

```
if( it_works(...) )    return true;
else                  return false;
```

- ne treba nam zapravo if ako je return jedina komanda

- `return it_works(...) ? true : false;`
- ne treba nam zapravo ternarni operator kad bukvalno imamo vrednost koja nam treba zapravo
 - `return it_works(...);`
- aritmetika
 - uvek sabiraj prvo najmanje cifre
 - $1.00 + .001 + .999 = ?$
 - $(1.00 + .999) + .001 = 1.99$
 - $(.001 + .999) + 1.00 = 2.00$
 - floatovi su pakao
 - greška gore iz prvog rezultata se zove *loss of significance*
 - - `double reordered_sum(double* from, double* upto) {
std :: sort(from, upto);
return std :: accumulate(from, upto, 0);
}`
 - Oops: The above starting value, 0, should be `double{ }!`
 - Oops 2: What if some (or all!) values are negative?
- zapravo ovako treba na kraju:
 - `auto less_in_magnitude = [] (double x, double y)
{ return std::abs(x) < std::abs(y); }`
 - `auto reordered_sum(double* from, double* upto) {
std::sort(from, upto, less_in_magnitude);
return std::accumulate(from, upto, double{ });
}`
- koristiti deltu za midpoint da se izbegne overflow
 - Possible approaches to implement `midpoint( a, b )`:
 - `return (a + b) / 2; ?`
 - `return a / 2 + b / 2; ?`
 - `return a + (b - a) / 2; ?` ←
 - od c++20 postoji `std::midpoint` da ne mora ovo da se piše
 - - `sqrt(x+1) - sqrt(x)` // *inaccurate when x > 1*
 - `1 / (sqrt(x+1) + sqrt(x))` // *equivalent, yet accurate for all x*
- postoje algoritmi tipa Kahan/Neumaier, pairwise, i slično za sabiranje
- [G.E. Forsythe: "Pitfalls in computation, or why a math book isn't enough."](#)